

Todo Application — Infrastructure & Deployment Notes

Author: Bhushan

Domain: *todo.bhusanshrestha.com.np* (Cloudflare subdomain)

1. Overview

This document records the full setup of the Todo application: provisioning the server with Terraform, installing the base stack, deploying the frontend and backend, configuring PostgreSQL and Nginx, hardening access, and setting up nightly backups.

Repositories:

- Frontend: github.com/bhushanportpro/todo-frontend
- Backend: github.com/bhushanportpro/todo-backend
- Deployment (source of truth for scripts/configs): github.com/bhushanportpro/todo-deployment
- Backend image: hub.docker.com/r/bhushanportpro/todo-app-backend (tag v1.0.0, semantic versioning)

2. Server Provisioning (Terraform)

The EC2 instance was provisioned with Terraform rather than the console, so the setup is reproducible and version-controlled.

2.1 Instance specification

Instance type: t3.medium

Root volume: 10 GB, gp3, encrypted with the account's portpro-kms KMS key (the account's default EBS key was unusable — Client.InvalidKMSKey.InvalidState)

AMI: Latest Ubuntu 24.04 LTS (looked up dynamically via the Canonical owner ID, filtered to hvm-ssd-gp3, x86_64, hvm, ebs)

Networking: Deployed into the existing Portpro-VPC and its public subnet (Portpro-VPC-public-us-east-1a) — no default VPC in this account

Elastic IP: Reserved and associated so the public IP doesn't change if the instance is replaced

SSH key: A dedicated key pair (terraform-ec2-key) created from the local `~/.ssh/id_ed25519.pub`

2.2 Security group

Only SSH access was opened initially — the instance was locked down to SSH-only until the app stack was ready, then HTTP/80 and HTTPS/443 were opened for the web-facing traffic. Outbound traffic is unrestricted.

Rule	Port	Protocol	Source
SSH	22	TCP	Restricted (public key only)
HTTP	80	TCP	0.0.0.0/0
HTTPS	443	TCP	0.0.0.0/0

2.3 Terraform outputs

The configuration outputs the instance ID, public IP, public DNS, and the Elastic IP so they can be fed into later steps (DNS, Ansible, CI/CD) without checking the console.

3. Base Software Install

Two setup scripts from the todo-deployment repo were run over SSH right after the instance came up:

```
bash nginx-docker.sh # installs nginx and Docker
bash postgresql.sh   # installs PostgreSQL
```

Keeping these as scripts in the deployment repo means the server bootstrap is repeatable and reviewable, instead of being a list of commands typed by hand.

4. Frontend Deployment

The frontend is a static export, served directly by Nginx rather than through Node.

```
git clone bhushanportpro/todo-frontend.git
cd todo-frontend
cp -r out /var/www/html
cd /var/www/html
rm -r default
```

The static build output (out/) is copied into /var/www/html, and Nginx's default site is removed so it can't be served by mistake. Ownership of /var/www/html is set to a dedicated frontend user rather than root or the docker user, keeping frontend and backend privileges separated (see Section 8).

5. Database Setup (PostgreSQL)

A dedicated database and a least-privilege application user were created rather than using the postgres superuser for the app:

```
psql -U postgres
CREATE DATABASE tododb;
CREATE USER todo WITH PASSWORD 'todo-access';
GRANT ALL PRIVILEGES ON DATABASE tododb TO todo;
GRANT USAGE ON SCHEMA public TO todo;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO todo;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO todo;
```

5.1 Network access for the containerized backend

Because the backend runs inside a Docker Swarm container, Postgres needed to accept connections from the Docker bridge network, not just localhost:

```
# /etc/postgresql/*/main/postgresql.conf
listen_addresses = '*'

# /etc/postgresql/*/main/pg_hba.conf
host tododb todo 172.18.0.0/16 scram-sha-256
```

Scoping the `pg_hba.conf` rule to the Docker bridge subnet (172.18.0.0/16) rather than 0.0.0.0/0 means only containers on that host's Docker network can reach Postgres, and authentication still requires `scram-sha-256`, not `trust`.

6. Backend Deployment (Docker Swarm)

The backend runs as a Docker Swarm service rather than a plain `docker run`, so it gets restart policies and rolling-update support for free.

```
mkdir todo-backend
docker pull bhushanportpro/todo-app-backend:v1.0.0
cd /tmp
cp docker-compose.yml /home/ubuntu/todo-backend
cp .env /home/ubuntu/todo-backend
cd /home/ubuntu/todo-backend
docker swarm init
docker stack deploy -c docker-compose.yml todo-backend
```

The `docker-compose.yml` and `.env` used for the stack are kept in the `todo-deployment` repo as the source of truth, so a redeploy doesn't depend on files sitting only on the server. The backend image itself follows semantic versioning (currently `v1.0.0`), so a deploy always pulls a known, pinned build rather than a floating latest tag.

7. Nginx Reverse Proxy & TLS

Nginx (`todo.conf`, from the `todo-deployment` repo) does three jobs: terminates TLS, serves the static frontend, and reverse-proxies `/api/` to the backend on 127.0.0.1:4000.

```
cp todo.conf /etc/nginx/sites-available/todo
ln -s /etc/nginx/sites-available/todo /etc/nginx/sites-enabled/
nginx -s reload
```

7.1 What the config enforces

- HTTP → HTTPS redirect on port 80; TLS termination on 443 via Let's Encrypt (Certbot-managed certs)
- Security headers on every response: HSTS, X-Frame-Options, X-Content-Type-Options, Referrer-Policy, Permissions-Policy, and a Content-Security-Policy restricted to 'self'
- A CORS allow-list (`map $http_origin $cors_origin`) that only reflects the production domain and localhost, instead of allowing every origin
- `/api/` proxied to the backend with the real client IP and protocol forwarded (`X-Real-IP`, `X-Forwarded-For`, `X-Forwarded-Proto`)
- Nginx basic auth (`auth_basic` + `htpasswd`) in front of the whole site as an extra access layer

```
htpasswd -c /etc/nginx/.htpasswd bhushan
```

8. Access Control & Hardening

Beyond the security group, access was hardened at the OS and application level:

- SSH is public-key only — password auth is disabled, so a leaked password alone can't get anyone in
- Two dedicated OS users, frontend and backend, follow least privilege instead of one shared account
- Directory ownership is scoped per user as needed — `/var/www/html` belongs to the frontend user; backend-related paths belong to the docker user — rather than everything running as root

- Additional users are created only as new access needs arise, keeping the permission set minimal over time

9. Operational Checks

9.1 Log rotation — verified, no action needed

Confirmed logrotate was already installed, configured, and running on its systemd timer before deciding not to add anything custom:

```
which logrotate
logrotate --version
cat /etc/logrotate.conf
ls /etc/logrotate.d/
systemctl list-timers | grep logrotate
cat /etc/cron.daily/logrotate
sudo cat /var/lib/logrotate/status
ls /var/log/
```

9.2 Confirming the app process

Checked that the backend was actually running as expected, both as a systemd/service unit and as an OS process, before trusting the deployment:

```
systemctl list-units --type=service --all | grep -i todo
ps aux | grep -i -E "node|python|gunicorn|todo|npm"
```

10. Automated Database Backups

A nightly pg_dump backup was scripted and scheduled rather than relying on manual dumps.

10.1 Backup script

```
#!/bin/bash
# Backup script for tododb

BACKUP_DIR="/opt/backups/postgres"
DATE=$(date +%Y-%m-%d_%H-%M-%S)
FILENAME="tododb_backup_${DATE}.sql.gz"
RETENTION_DAYS=7

mkdir -p "$BACKUP_DIR"

# Dump the database and compress it
sudo -u postgres pg_dump tododb | gzip > "$BACKUP_DIR/$FILENAME"

# Delete backups older than RETENTION_DAYS
find "$BACKUP_DIR" -name "tododb_backup_*.sql.gz" -mtime +$RETENTION_DAYS -delete

echo "Backup completed: $FILENAME" >> "$BACKUP_DIR/backup.log"
```

Backups older than 7 days are pruned on every run, so at steady state the server holds 7 nightly backups at a time.

10.2 Setup and scheduling

```
sudo mkdir -p /opt/backups
sudo nano /opt/backups/backup_tododb.sh
sudo chmod +x /opt/backups/backup_tododb.sh
sudo nice -n 19 /opt/backups/backup_tododb.sh # manual test run
ls -lh /opt/backups/postgres/
cat /opt/backups/postgres/backup.log
sudo crontab -e
```

Cron line added (runs nightly at midnight, with nice -n 19 so the dump doesn't compete for CPU with the running app):

```
0 0 * * * /usr/bin/nice -n 19 /opt/backups/backup_tododb.sh >> /opt/backups/cron.log 2>&1
```

11. Repositories, Images & Versioning

Component	Location	Notes
Frontend	github.com/bhushanportpro/todo-frontend	Static export, served by Nginx
Backend	github.com/bhushanportpro/todo-backend	Source for the backend image
Deployment	github.com/bhushanportpro/todo-deployment	Source of truth: compose file, nginx-docker.sh, postgresql.sh, todo.conf
Backend image	hub.docker.com/r/bhushanportpro/todo-app-backend	Tagged v1.0.0, semantic versioning

Access to push images/code was initially kept to a single account; it was later opened up so the image and repos can be pushed to from other machines as well.

12. Summary

The stack now runs as: Terraform-provisioned EC2 (Ubuntu 24.04, t3.medium, encrypted gp3 root volume, Elastic IP) → Nginx terminating TLS and serving the frontend → Nginx reverse-proxying /api/ to a Dockerized backend running under Docker Swarm → PostgreSQL restricted to the Docker bridge subnet → nightly backups with a 7-day retention window, all fronted by SSH key-only access, least-privilege OS users, and basic auth plus a locked-down CSP/CORS policy at the edge.